GenomicFeatures

Kasper D. Hansen

- Dependencies
- Corrections
- Overview
- Other Resources
- Examples
- Caution: Terminology
- Gene, exons and transcripts
- SessionInfo

Dependencies

This document has the following dependencies:

```
library(GenomicFeatures)
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicFeatures", "TxDb.Hsapiens.UCSC.hg19.knownGene"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

The *GenomicFeatures* package contains functionality for so-called transcript database or *TxDb* objects. These objects contains a coherent interface to transcripts. Transcripts are complicated because higher organisms usually have many different transcripts for each gene locus.

Other Resources

- The vignette from the GenomicFeatures webpage.

Examples

We will show the `TxDb` functionality by examining a database of human transcripts. Unlike genomes in Bioconductor, there is no shorthand object; we reassign the long name to a shorter for convenience:

```
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
txdb <- TxDb.Hsapiens.UCSC.hg19.knownGene
txdb
```

```
## TxDb object:
## # Db type: TxDb
## # Supporting package: GenomicFeatures
## # Data source: UCSC
## # Genome: hg19
## # Organism: Homo sapiens
## # UCSC Table: knownGene
## # Resource URL: http://genome.ucsc.edu/
## # Type of Gene ID: Entrez Gene ID
## # Full dataset: yes
## # miRBase build ID: GRCh37
## # transcript_nrow: 82960
## # exon_nrow: 289969
## # cds_nrow: 237533
## # Db created by: GenomicFeatures package from Bioconductor
## # Creation time: 2015-03-19 13:55:51 -0700 (Thu, 19 Mar 2015)
## # GenomicFeatures version at creation time: 1.19.32
## # RSQLite version at creation time: 1.0.0
## # DBSCHEMAVERSION: 1.1
```

A `TxDb` object is really an interface to a `SQLite` database. You can query the database using a number of tools detailed in the package vignette, but usually you use convenience functions to extract the relevant information.

Extract basic quantities

- `genes()`
- `transcripts()`

- `cds()`
- `exons()`
- `microRNAs()`
- `tRNAs()`
- `promoters()`

Extract quantities and group

- `transcriptsBy(by = c("gene", "exon", "cds"))`
- `cdsBy(by = c("tx", "gene"))`
- `exonsBy(by = c("tx", "gene"))`
- `intronsByTranscript()`
- `fiveUTRsByTranscript()`
- `threeUTRsByTranscript()`

(Note: there are grouping functions without the non-grouping function and vice versa; there is for example no `introns()` function.

Other functions

- `transcriptLengths()` (optionally include CDS length etc).
- `XXByOverlaps()` (select features based on overlaps with `XX` being `transcript`, `cds` or `exon`).

Mapping between genome and transcript coordinates

- `extractTranscriptSeqs()` (getting RNA sequencing of the transcripts).

Caution: Terminology

The `TxDb` object approach is powerful but it suffers (in my opinion) from a lack of clearly defined terminology. Even worse, the meaning of terminology changes depending on the function. For example `transcript` is sometimes used to refer to un-spliced transcripts (pre-mRNA) and sometimes to splices transcripts.

Gene, exons and transcripts

Let us start by examining genes, exons and transcripts. Let us focus on a single gene on chr1: DDX11L1.

```
gr <- GRanges(seqnames = "chr1", strand = "+", ranges = IRanges(start =
subsetByOverlaps(genes(txdb), gr)
```

```
## GRanges object with 1 range and 1 metadata column:
##             seqnames         ranges strand |     gene_id
##                <Rle>      <IRanges>  <Rle> | <character>
##   100287102     chr1 [11874, 14409]      + |   100287102
##   -------
##   seqinfo: 93 sequences (1 circular) from hg19 genome
```

```
subsetByOverlaps(genes(txdb), gr, ignore.strand = TRUE)
```

```
## GRanges object with 2 ranges and 1 metadata column:
##             seqnames         ranges strand |     gene_id
##                <Rle>      <IRanges>  <Rle> | <character>
##   100287102     chr1 [11874, 14409]      + |   100287102
##      653635     chr1 [14362, 29961]      - |      653635
##   -------
##   seqinfo: 93 sequences (1 circular) from hg19 genome
```

The `genes()` output contains a single gene with these coordinates, overlapping another gene on the opposite strand. Note that the gene is represented as a single range; so this output tells us nothing about exons and splicing. There is a single identifier called `gene_id`. If you look at the output of `txdb` you'll see that this is an "Entrex Gene ID".

```
subsetByOverlaps(transcripts(txdb), gr)
```

```
## GRanges object with 3 ranges and 2 metadata columns:
##       seqnames         ranges strand |     tx_id     tx_name
##          <Rle>      <IRanges>  <Rle> | <integer> <character>
##   [1]     chr1 [11874, 14409]      + |         1  uc001aaa.3
##   [2]     chr1 [11874, 14409]      + |         2  uc010nxq.1
##   [3]     chr1 [11874, 14409]      + |         3  uc010nxr.1
##   -------
##   seqinfo: 93 sequences (1 circular) from hg19 genome
```

The gene has 3 transcripts; again we only have coordinates of the pre-mRNA here. There are 3 different transcript names (`tx_name`) which are identifiers from UCSC and then we have a `TxDb` specific transcript id (`tx_id`) which is an integer. Let's look at exons:

```
subsetByOverlaps(exons(txdb), gr)
```

```
## GRanges object with 6 ranges and 1 metadata column:
##       seqnames          ranges strand |   exon_id
##          <Rle>       <IRanges>  <Rle> | <integer>
##   [1]     chr1 [11874, 12227]      + |         1
##   [2]     chr1 [12595, 12721]      + |         2
##   [3]     chr1 [12613, 12721]      + |         3
##   [4]     chr1 [12646, 12697]      + |         4
##   [5]     chr1 [13221, 14409]      + |         5
##   [6]     chr1 [13403, 14409]      + |         6
##   -------
##   seqinfo: 93 sequences (1 circular) from hg19 genome
```

Here we get 6 exons, but no indication of which exons makes up which transcripts. To get this, we can do

```
subsetByOverlaps(exonsBy(txdb, by = "tx"), gr)
```

```
## GRangesList object of length 3:
## $1
## GRanges object with 3 ranges and 3 metadata columns:
##       seqnames         ranges strand |   exon_id   exon_name exon_ran
##          <Rle>      <IRanges>  <Rle> | <integer> <character> <integer
##   [1]     chr1 [11874, 12227]      + |         1        <NA>
##   [2]     chr1 [12613, 12721]      + |         3        <NA>
##   [3]     chr1 [13221, 14409]      + |         5        <NA>
##
## $2
## GRanges object with 3 ranges and 3 metadata columns:
##       seqnames         ranges strand | exon_id exon_name exon_rank
##   [1]     chr1 [11874, 12227]      + |       1      <NA>         1
##   [2]     chr1 [12595, 12721]      + |       2      <NA>         2
##   [3]     chr1 [13403, 14409]      + |       6      <NA>         3
##
## $3
## GRanges object with 3 ranges and 3 metadata columns:
##       seqnames         ranges strand | exon_id exon_name exon_rank
##   [1]     chr1 [11874, 12227]      + |       1      <NA>         1
##   [2]     chr1 [12646, 12697]      + |       4      <NA>         2
##   [3]     chr1 [13221, 14409]      + |       5      <NA>         3
##
## -------
## seqinfo: 93 sequences (1 circular) from hg19 genome
```

Here we now finally see the structure of the three transcripts in the form of a `GRangesList`.

Let us include the coding sequence (CDS). Now, it can be extremely hard to computationally infer the coding sequence from a fully spliced mRNA.

```
subsetByOverlaps(cds(txdb), gr)
```

```
## GRanges object with 3 ranges and 1 metadata column:
##       seqnames         ranges strand |    cds_id
##          <Rle>      <IRanges>  <Rle> | <integer>
##   [1]     chr1 [12190, 12227]      + |         1
##   [2]     chr1 [12595, 12721]      + |         2
##   [3]     chr1 [13403, 13639]      + |         3
##   -------
##   seqinfo: 93 sequences (1 circular) from hg19 genome
```

```
subsetByOverlaps(cdsBy(txdb, by = "tx"), gr)
```

```
## GRangesList object of length 1:
## $2
## GRanges object with 3 ranges and 3 metadata columns:
##       seqnames          ranges strand |    cds_id    cds_name exon_ran
##          <Rle>       <IRanges>  <Rle> | <integer> <character> <integer
##   [1]     chr1 [12190, 12227]      + |         1        <NA>
##   [2]     chr1 [12595, 12721]      + |         2        <NA>
##   [3]     chr1 [13403, 13639]      + |         3        <NA>
##
## -------
## seqinfo: 93 sequences (1 circular) from hg19 genome
```

The output of `cds()` is not very useful by itself, since each range is part of a CDS, not the entire cds. We need to know how these ranges together form a CDS, and for that we need `cdsBy(by = "tx")`. We can see that only one of the three transcripts has a CDS by looking at their CDS lengths:

```
subset(transcriptLengths(txdb, with.cds_len = TRUE), gene_id == "1002871
```

```
##   tx_id    tx_name   gene_id nexon tx_len cds_len
## 1     1 uc001aaa.3 100287102     3   1652       0
## 2     2 uc010nxq.1 100287102     3   1488     402
## 3     3 uc010nxr.1 100287102     3   1595       0
```

(here we subset a `data.frame`).

Note: as an example of terminology mixup, consider that the output of `transcripts()` are coordinates for the unspliced transcript, whereas `extractTranscriptSeqs()` is the RNA sequence of the spliced transcripts.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
##  [1] XVector_0.8.0
##  [2] TxDb.Hsapiens.UCSC.hg19.knownGene_3.1.2
##  [3] GenomicFeatures_1.20.2
##  [4] AnnotationDbi_1.30.1
##  [5] Biobase_2.28.0
##  [6] GenomicRanges_1.20.5
##  [7] GenomeInfoDb_1.4.2
##  [8] IRanges_2.2.7
##  [9] S4Vectors_0.6.3
## [10] BiocGenerics_0.14.0
## [11] BiocStyle_1.6.0
## [12] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] knitr_1.11              magrittr_1.5
##  [3] GenomicAlignments_1.4.1 zlibbioc_1.14.0
##  [5] BiocParallel_1.2.20     stringr_1.0.0
##  [7] tools_3.2.1             DBI_0.3.1
##  [9] lambda.r_1.1.7          futile.logger_1.4.1
## [11] htmltools_0.2.6         yaml_2.1.13
## [13] digest_0.6.8            rtracklayer_1.28.8
## [15] formatR_1.2             futile.options_1.0.0
## [17] bitops_1.0-6            RCurl_1.95-4.7
## [19] biomaRt_2.24.0          evaluate_0.7.2
## [21] RSQLite_1.0.0           stringi_0.5-5
## [23] Rsamtools_1.20.4        Biostrings_2.36.3
## [25] XML_3.98-1.3
```